东南大学

BJSL0022 - 程序设计基础及语言 I - 12

2026-2027 学年秋季学期

Lab2 类、对象和字符串的介绍

截止日期: 2026 年 11 月 1 日（周日）17：59

学习目标：

1. 如何定义类、如何用类创建对象实例
2. 如何将类的行为实现为成员函数
3. 如何将类的属性实现为数据成员
4. 如何调用对象实例的成员函数执行任务
5. 类的数据成员与函数的局部变量的不同之处
6. 如何使用构造函数初始化对象实例的数据
7. 如何精心设计类，使它的实现与接口分离并有利于复用
8. 如何创建和使用`std::string`的对象实例

实验：

Ex1: (P.76，3.15）创建一个 Date 类，用于表示年月日，具体要求如下：

(a) 它包含三个数据成员：`year, month, day`，三者均为`int`类型
(b) 它有一个 3 个形参的构造函数，用于初始化上述的三个成员
(c) 对于每个成员，提供一组用于设置的 set 函数和用于获取的 get 函数
(d) 可以假设年和日的值都是正确的，但月的值可能不正确。若超出范围，将其设置为 1
(e) 提供成员函数`displayDate()`，显示用斜杠（/）分割的年/月/日格式
(d) 编写测试程序，演示`Date`类的作用

```
(输出样例)
Input year: 2026
Input month: 13
Input day: 10
The date is: 2026/1/10
```

```
// Date.h
class Date {
public:
    /* TODO: consturctor function prototype */
    /* TODO: setter and getter function prototype */
    /* TODO: display function prototype */
private:
    /* TODO: data members */
};
```

```cpp
// Date.cpp
#include "Date.h"
#include <iostream>
using namespace std;

/* TODO: constructor function definition */
/* TODO: setter and getter function definition */
/* TODO: display function definition */
```

```cpp
// ex1.cpp
#include <iostream>
#include "Date.h"
using namespace std;

int main() {
    int year, month, day;
    cout << "Input year: "; cin >> year;
    cout << "Input month: "; cin >> month;
    cout << "Input day: "; cin >> day;
    /* TODO: create an object of class Date */
    cout << "The date is: ";
    /* TODO: diplay the date by calling member function */
    cout << endl;
}
```

问题解决技巧：

1. 将数据成员定义为`private`，通过 set 与 get 函数来控制访问
2. 构造函数中使用 set 避免代码重复
3. 构造测试用例时，应尽可能的考虑边界或例外情况

思考问题：

1. 尝试按照标准格式，即 “yyyy-mm-dd” 输出固定长度的日期，假设年份合法
2. 尝试写一个`int compare(Date d)`函数，用于和另一个日期比较大小。规则如下：如果二者日期相同，则返回 0；如果当前日期早于参数日期，返回-1；否则返回 1。

```
(输出样例)
Date1 is: 2026/1/10
Date2 is: 2026/10/1
Date1 is earlier than Date2
```

Ex2: 某银行系统拟使用`Account`类储存用户账户余额。该类包含用户名称与整数余额，可通过构造函数进行初始化。任何时候，用户余额都至少为 0，若初始化时小于 0，应设置为 0 并输出错误信息。该类应支持三项操作：

(a) `display`展示用户名及余额

(b) `deposit`存款函数，提示用户输入存款金额，并增加余额

(c) `withdraw`取款函数，提示用户输入金额，若余额充足则减少余额，否则不变并输出错误信息

```
(输出样例)
account1 balance: $50
account2 balance: $25

Enter deposit amount for account1: 20
Added 20 to account1, new balance is: $70

Enter withdrawal amount for account1: 15
Withdrew 15 from account1, new balance is: $55

Enter withdrawal amount for account2: 30
[Error] Operation refused: debit amount exceeded account balance!

account1 balance: $55
account2 balance: $25
```

```h
// Account.h

/* TODO: your code for the interface */
```

```cpp
// Account.cpp
#include "Account.h"
#include <iostream>
using namespace std;

/* TODO: your code for other required functions */

void Account::display() {
    cout << getName() << " balance: $" << getBalance() << endl;
}

void Account::deposit() {
    cout << "Enter deposit amount for " << getName() << ": ";
    unsigned amount; cin >> amount;

    /* TODO: your code for deposit logic */

    cout << "Added " << amount << " to " << getName()
         << ", new balance is $" << getBalance() << endl;
}

void Account::withdraw() {
    cout << "Enter withdrawal amount for " << getName() << ": ";
    unsigned amount; cin >> amount;

    /* TODO: your code for withdraw logic */

    cout << "Withdrew " << amount << " from " << getName()
         << ", new balance is $" << getBalance() << endl;
}
```

```cpp
// ex2.cpp
#include <iostream>
#include "Account.h"
using namespace std;

int main() {
    /* TODO: creating objects */
    a1.display();
    a2.display();
    cout << endl;

    a1.deposit(); cout << endl;
    a1.withdraw(); cout << endl;
    a2.withdraw(); cout << endl;

    a1.display();
    a2.display();
}
```

问题解决技巧：

1. 根据问题描述设计函数原型，写入头文件`Account.h`
2. 根据函数原型需求，将函数实现写入源文件`Account.cpp`
3. 在`ex2.cpp`中写入`main`函数，对功能进行测试

思考问题：

1. 尝试实现从一个用户向另一用户转账的函数，即`a1.transfer(a2, 10)`
2. 你实现的函数是否成功减少了 a1 的余额？是否成功增加了 a2 的余额？为什么？